

practical5

StudentTest.java

```
// Import Student: the class from the studentinfo package that holds the student details
// and provides the methods to accept, display and grade the data. Because StudentTest is
// placed outside the studentinfo package, it must be imported to be usable here.
import studentinfo.Student;

/**
 * StudentTest class placed outside the studentinfo package.
 * Imports the studentinfo.Student class, accepts details and
 * displays the student information along with the grade.
 */
public class StudentTest {
    public static void main(String[] args) {
        // Create a new Student object. All its fields start with default values
        // (0 / null) until acceptDetails() fills them in.
        Student student = new Student();

        // Ask the user to type in the student details and store them inside the student object.
        student.acceptDetails();

        // Print a blank line to keep the output tidy.
        System.out.println();
        // Print a heading so the user knows the student information is coming next.
        System.out.println("--- Student Information ---");
        // Display all the details that were accepted earlier.
        student.displayDetails();
        // Print the grade (e.g. A+) calculated from the marks stored in the student object.
        System.out.println("Grade: " + student.getGrade());
    }
}
```

Q2Test.java

```
// Import Employee: the interface from the company package that declares the calculateSalary() contract.
// Importing it lets the test class refer to any worker through the interface type.
import company.Employee;
// Import Manager: the class from the department package that implements the company.Employee interface.
import department.Manager;

/**
 * Demonstrates importing an interface from one package (company)
 * and using a class that implements it from another package (department).
 */
public class Q2Test {
    public static void main(String[] args) {
        // Create a Manager object but hold it inside an Employee interface reference.
        // This works because Manager implements Employee (managers ARE employees).
        Employee employee = new Manager("Harkeerat", 60000, 15000);
        // Call the calculateSalary() method through the interface reference.
        // Java runs the Manager's version at runtime (dynamic method dispatch).
        employee.calculateSalary();
    }
}
```

Q3_ManagerDeveloper.java

```
// Import Scanner: used to read text and numbers typed on the keyboard by the user.
import java.util.Scanner;

/**
 * Employee interface declaring the common operations for all employees.
 * Any class representing a worker (Manager, Developer, etc.) must
 * implement these two methods.
 */
interface Employee {
    /**
     * Calculates the total salary of the employee.
     */
    void calculateSalary();

    /**
     * Displays the personal and payroll details of the employee.
     */
    void displayDetails();
}

/**
 * Manager class implementing the Employee interface.
 */
class Manager implements Employee {
    // Stores the name of the manager.
    String name;
    // Stores the fixed basic salary of the manager.
    double basicSalary;
    // Stores the bonus amount on top of the basic salary.
    double bonus;

    /**
     * Constructor: Initializes a Manager with a name, basic salary and bonus.
     * @param name the manager's name
     * @param basicSalary the fixed basic salary
     * @param bonus the bonus amount
     */
    Manager(String name, double basicSalary, double bonus) {
        // Store the passed-in name into the manager's field.
        this.name = name;
        // Store the passed-in basic salary into the manager's field.
        this.basicSalary = basicSalary;
        // Store the passed-in bonus into the manager's field. 'this' tells the
        // field apart from the parameter with the same name.
        this.bonus = bonus;
    }

    @Override
    public void calculateSalary() {
        // Print the total salary, which is basic salary plus bonus.
        System.out.println("Salary: " + (basicSalary + bonus));
    }

    @Override
    public void displayDetails() {
        // Print the job designation first as a heading.
        System.out.println("Designation: Manager");
        // Print the manager's name.
        System.out.println("Name: " + name);
    }
}
```

```

        // Print the manager's basic salary.
        System.out.println("Basic Salary: " + basicSalary);
        // Print the manager's bonus.
        System.out.println("Bonus: " + bonus);
    }
}

/**
 * Developer class implementing the Employee interface.
 */
class Developer implements Employee {
    // Stores the name of the developer.
    String name;
    // Stores the fixed basic salary of the developer.
    double basicSalary;
    // Stores the number of projects the developer has completed.
    int projects;

    /**
     * Constructor: Initializes a Developer with a name, basic salary and project count.
     * @param name the developer's name
     * @param basicSalary the fixed basic salary
     * @param projects the number of projects completed
     */
    Developer(String name, double basicSalary, int projects) {
        // Store the passed-in name into the developer's field.
        this.name = name;
        // Store the passed-in basic salary into the developer's field.
        this.basicSalary = basicSalary;
        // Store the passed-in project count into the developer's field.
        this.projects = projects;
    }

    @Override
    public void calculateSalary() {
        // Each completed project adds a fixed 5000 bonus amount.
        double projectBonus = projects * 5000;
        // Print the total salary, which is basic salary plus the project bonus.
        System.out.println("Salary: " + (basicSalary + projectBonus));
    }

    @Override
    public void displayDetails() {
        // Print the job designation first as a heading.
        System.out.println("Designation: Developer");
        // Print the developer's name.
        System.out.println("Name: " + name);
        // Print the developer's basic salary.
        System.out.println("Basic Salary: " + basicSalary);
        // Print the number of projects the developer has completed.
        System.out.println("Projects Completed: " + projects);
    }
}

/**
 * Accepts employee details and salary from the user and displays
 * the details and salary for each type of employee.
 */
public class Q3_ManagerDeveloper {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
    }
}

```

```

// Print a heading to show we are entering the manager section.
System.out.println("--- Enter Manager Details ---");
// Prompt the user for the manager name.
System.out.print("Name: ");
// Read the whole name line and store it in mName.
String mName = sc.nextLine();
// Prompt the user for the basic salary.
System.out.print("Basic Salary: ");
// Read the decimal basic salary and store it in mBasic.
double mBasic = sc.nextDouble();
// Prompt the user for the bonus.
System.out.print("Bonus: ");
// Read the decimal bonus and store it in mBonus.
double mBonus = sc.nextDouble();
// Build a Manager object from the entered values.
Manager manager = new Manager(mName, mBasic, mBonus);

// Consume the leftover newline so the next nextLine() is not skipped.
sc.nextLine();
// Print a heading to show we are entering the developer section.
System.out.println("\n--- Enter Developer Details ---");
// Prompt the user for the developer name.
System.out.print("Name: ");
// Read the whole name line and store it in dName.
String dName = sc.nextLine();
// Prompt the user for the basic salary.
System.out.print("Basic Salary: ");
// Read the decimal basic salary and store it in dBasic.
double dBasic = sc.nextDouble();
// Prompt the user for the number of projects.
System.out.print("Number of Projects: ");
// Read the integer project count and store it in projects.
int projects = sc.nextInt();
// Build a Developer object from the entered values.
Developer developer = new Developer(dName, dBasic, projects);

// Print a heading for the manager's output.
System.out.println("\n--- Manager ---");
// Show the manager's personal and payroll details.
manager.displayDetails();
// Show the manager's total salary.
manager.calculateSalary();

// Print a heading for the developer's output.
System.out.println("\n--- Developer ---");
// Show the developer's personal and payroll details.
developer.displayDetails();
// Show the developer's total salary.
developer.calculateSalary();
}
}

```

Q4_BankAccount.java

```
// Import Scanner: used to read text and numbers typed on the keyboard by the user.
import java.util.Scanner;

/**
 * Account interface declaring the bank operations.
 * Any bank account class (e.g. SavingsAccount) must implement
 * these three operations.
 */
interface Account {
    /**
     * Adds money to the account balance.
     * @param amount the amount to deposit
     */
    void deposit(double amount);

    /**
     * Removes money from the account balance.
     * @param amount the amount to withdraw
     */
    void withdraw(double amount);

    /**
     * Prints the current account balance.
     */
    void checkBalance();
}

/**
 * SavingsAccount class implementing the Account interface.
 */
class SavingsAccount implements Account {
    // Stores the current money available in the account.
    double balance;

    /**
     * Constructor: Initializes the account with a starting balance.
     * @param initialBalance the balance the account starts with
     */
    SavingsAccount(double initialBalance) {
        // Store the starting balance into the account's balance field.
        balance = initialBalance;
    }

    @Override
    public void deposit(double amount) {
        // Add the deposited amount to the current balance.
        balance += amount;
        // Confirm the deposit by printing the amount that was added.
        System.out.println("Deposited: " + amount);
    }

    @Override
    public void withdraw(double amount) {
        // If the requested amount is more than what is available...
        if (amount > balance) {
            // ...reject the withdrawal and tell the user the balance is insufficient.
            System.out.println("Insufficient balance. Withdrawal failed.");
        } else {
            // Otherwise subtract the withdrawn amount from the balance.
        }
    }
}
```

```

        balance -= amount;
        // Confirm the withdrawal by printing the amount that was taken out.
        System.out.println("Withdrawn: " + amount);
    }
}

@Override
public void checkBalance() {
    // Print the current money available in the account.
    System.out.println("Current Balance: " + balance);
}

}

/**
 * Accepts the initial balance from the user and allows deposit,
 * withdrawal and balance enquiry operations.
 */
public class Q4_BankAccount {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Prompt the user for the opening balance.
        System.out.print("Enter initial balance: ");
        // Read the decimal value typed by the user and store it in initial.
        double initial = sc.nextDouble();
        // Create a savings account starting with the entered balance.
        SavingsAccount account = new SavingsAccount(initial);

        // Will hold the menu option chosen by the user (loop control variable).
        int choice;
        // do-while loop: the menu is shown at least once and repeats
        // until the user picks option 4 (Exit).
        do {
            // Print the bank menu heading.
            System.out.println("\n--- Bank Account Menu ---");
            // Show the four available options.
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Check Balance");
            System.out.println("4. Exit");
            // Prompt the user to pick an option.
            System.out.print("Enter your choice: ");
            // Read the chosen option number and store it in choice.
            choice = sc.nextInt();

            // Run the selected operation based on the chosen number.
            switch (choice) {
                case 1:
                    // Deposit option: ask how much money to add.
                    System.out.print("Enter amount to deposit: ");
                    // Read the amount and pass it straight to the deposit() method.
                    account.deposit(sc.nextDouble());
                    // Leave the switch after handling this option.
                    break;
                case 2:
                    // Withdraw option: ask how much money to take out.
                    System.out.print("Enter amount to withdraw: ");
                    // Read the amount and pass it straight to the withdraw() method.
                    account.withdraw(sc.nextDouble());
                    // Leave the switch after handling this option.
                    break;
                case 3:

```

```
        // Balance option: show the current account balance.
        account.checkBalance();
        // Leave the switch after handling this option.
        break;
    case 4:
        // Exit option: print a goodbye message to the user.
        System.out.println("Thank you for using the bank account system.");
        // Leave the switch after handling this option.
        break;
    default:
        // Any number other than 1-4 is rejected as invalid.
        System.out.println("Invalid choice.");
    }
    // Keep looping while the user has not chosen option 4 (Exit).
} while (choice != 4);
}
```